# MOBILE DEVELOPMENT

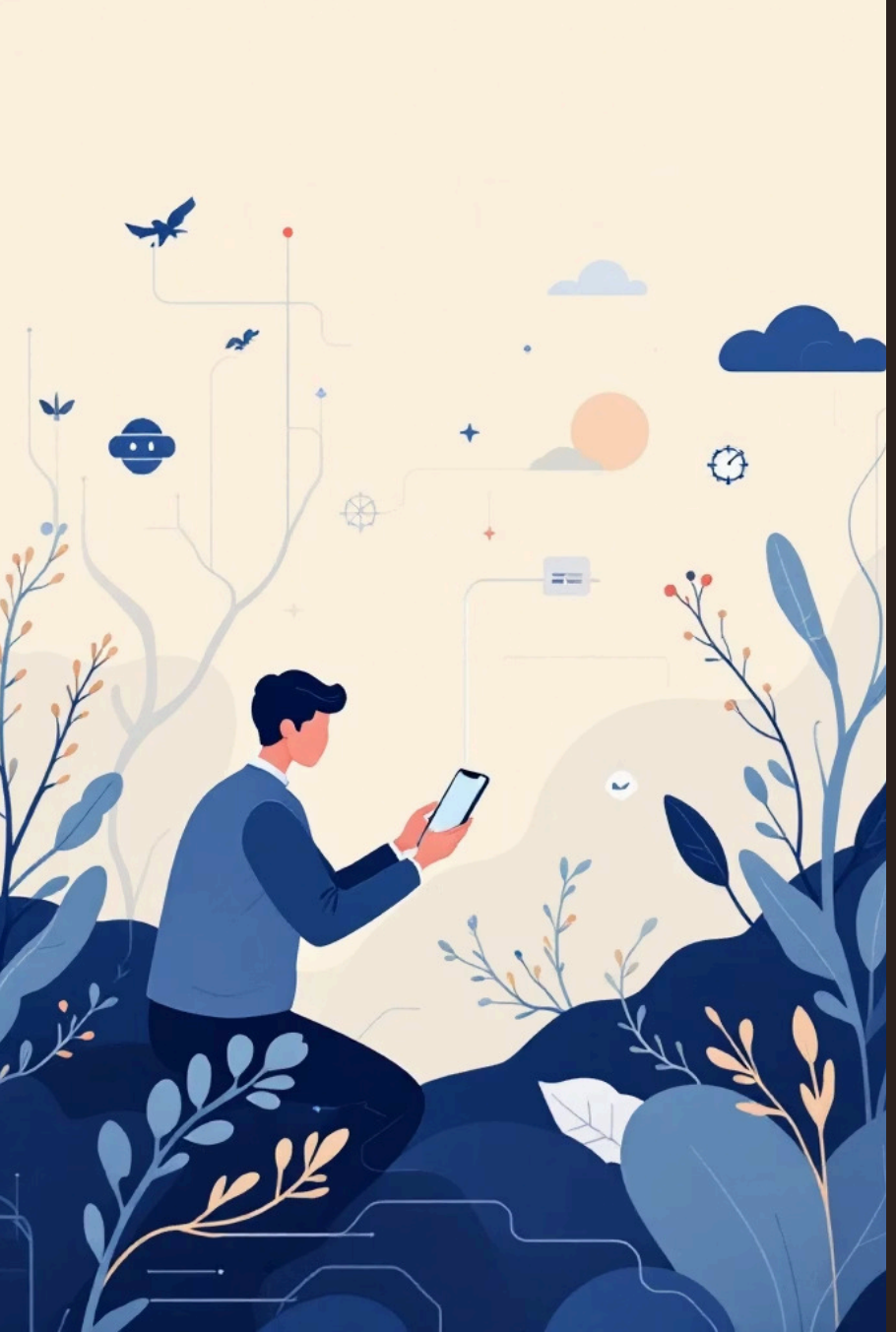
## Android / iOS with Flutter

**Renel Sainfort**
ESILV – Engineering School

### CM4: Connect : Building Connected Apps

# SESSION

01 — Introduction
02 — The 5 Connection Families
03 — Model Context Protocol (MCP)
04 — What's next ?



Chapter 01

# Introduction

# CM3 Retrospective

## Clean Architecture

*The 5 Flutter Layers*

- Model
- Repository
- Service
- Controller
- Screen

## 3 documents system

*BRIEF_PO.md / ARCHITECTURE.md / DESIGN_SYSTEM.m*

1. **BRIEF_PO.md**
   Screens, actions, data
2. **ARCHITECTURE.md**
   Structure and responsibilities
3. **DESIGN_SYSTEM.md**
   Colors, typography, spacing

# Limitation

Without connectivity, a mobile application remains functional but cannot scale beyond its own boundaries.

# Central Hypothesis

In 2026, the power of a mobile application lies in its ability to be connected, integrated and contextual – not in its complexity.

Chapter 02

# The 5 Connection Families

# The 5 Connection Families

### Data

Store and sync your app's data

### Identity

Identify the user and control their access

### API

Consume external services via HTTP

### Realtime

React instantly to data changes

### IoT

Connect an app to physical objects

# Data — Local

Store data directly on the device, without an internet connection.

## Tools

| Tool | Description |
|---|---|
| Hive | Lightweight, fast, key-value store – ideal for preferences and settings |
| Isar | Relational, complex queries, high performance |

## Use Cases

### Duolingo

Progress saved offline

### Google Maps

Maps available without internet

# Data — Cloud

Store and sync your app's data in the cloud.

## Tools

| Tool | Description |
|---|---|
| Firebase Firestore | NoSQL, native real-time, mature Flutter SDK, automatic offline sync |
| Supabase | SQL, open-source, auto-generated API, self-hostable |

## Use Cases

### WhatsApp

Messages synced between users

### Notion

Collaborative documents with user relationships

# Identity

Identify the user and control their access.

## Tools

| Tool | Description |
|---|---|
| Firebase Auth | Native Flutter, coupled with Firestore, built-in Google Sign-In |
| Supabase Auth | Integrated with Supabase, OAuth providers, same logic as Firebase Auth |
| Clerk | Advanced role and permission management, multi-tenant SaaS oriented |

## Use Cases

### Notion

Email sign-up, access to personal documents

### Slack

Workspace members with different roles

# API

Consume an external service from your app via HTTP requests.

## HTTP Clients

| Tool | Description | Level |
|---|---|---|
| http | Official HTTP client, lightweight and simple | POC / small projects |
| Dio | Advanced HTTP client – interceptors, timeouts, error handling, auth | ✅ Recommended MVP |
| Chopper | Strongly typed REST client, service generation | Advanced – MVP overkill |

## Use Cases

### Stripe

Process a payment

### OpenWeather

Real-time weather

# Realtime

React instantly to data changes without the user needing to refresh the app.

## Tools

| Tool | Description |
| --- | --- |
| Firebase Firestore | Live synchronization of NoSQL documents across all connected devices |
| Supabase Realtime | Listens to PostgreSQL changes via WebSockets |

## Use Cases

### Airbnb

Real-time property availability

### Twitch

Live viewer count

# IoT

Connect your app to physical objects via Bluetooth, NFC, or WiFi.

## Tools

| Tool | Description |
|---|---|
| flutter_blue_plus | Communicate with objects via Bluetooth Low Energy (BLE) |
| nfc_manager | Read and write NFC tags from Flutter |

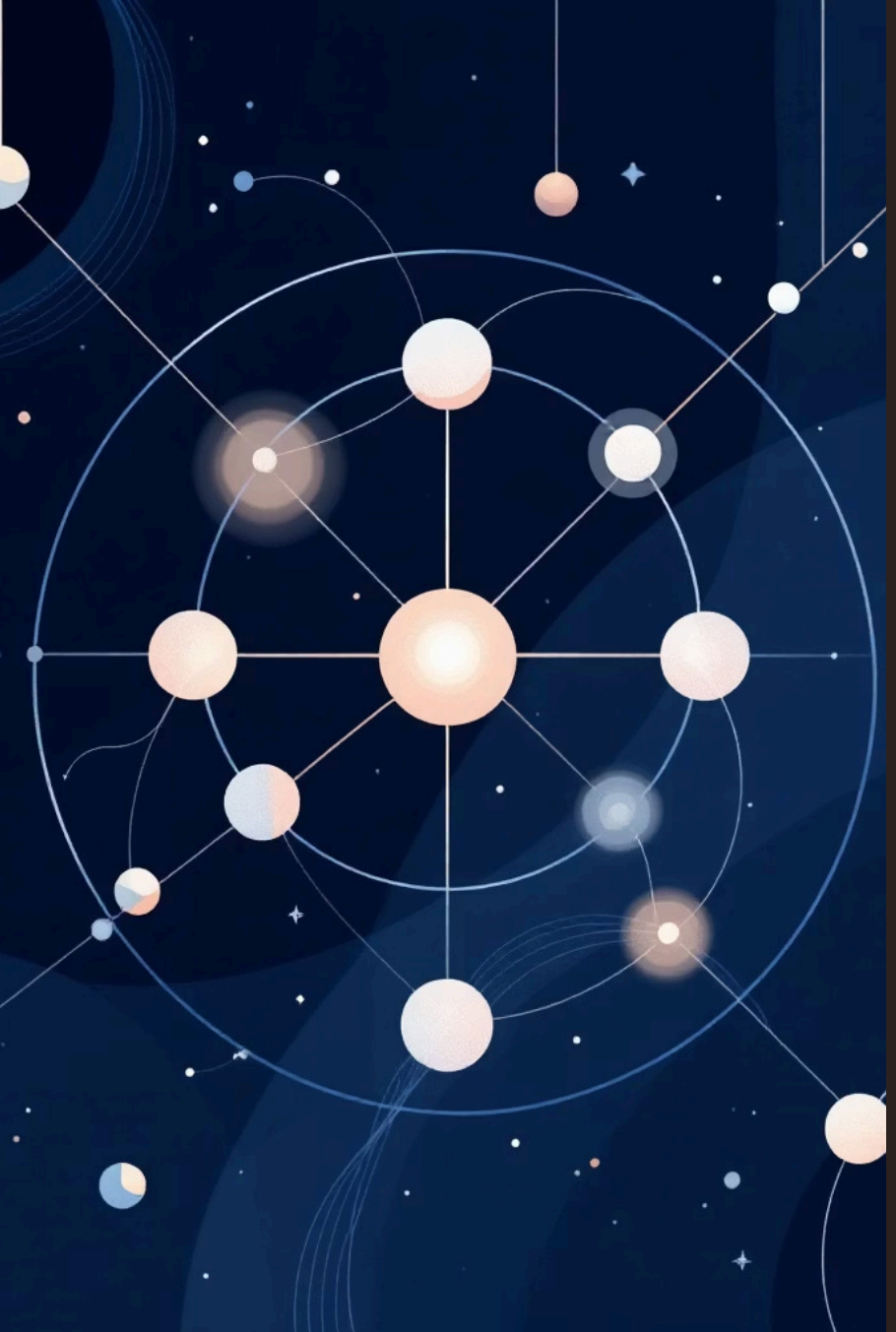
## Use Cases

### Apple Watch

Heart rate data via Bluetooth

### Navigo

Top up your transit pass via NFC



CHAPTER 03

# Model Context Protocol (MCP)

# How can my AI agent act on external tools, without leaving my editor?

## Before Agentic Code

- The developer does everything manually
- Configures each tool in the browser
- Copy-pastes keys into the project
- Writes all the code alone
- Juggles between multiple environments

## Before MCP

- The AI agent suggests code but cannot act
- Custom middleware for each action
- Create a table → 1 middleware
- Configure Auth → 1 middleware
- N actions × M tools = hundreds of middlewares

# MCP — Model Context Protocol

An open standard that allows an AI agent to connect and interact with external tools and services in a standardized way.

### Created by

Anthropic

### Released

November 2024

### Adopted by

OpenAI, Google DeepMind – Donated to the Linux Foundation, December 2025

# MCP — Server & Client

## MCP Client

The AI agent that consumes MCP Servers

*"I need to do this"*

- Claude Code
- Gemini CLI
- Cursor
- VS Code Copilot

## MCP Protocol

The common language between Client and Server

*Open standard – any client speaks to any server*

## MCP Server

The external service that exposes its capabilities

*"Here is what I can do"*

- Firebase MCP Server
- Stripe MCP Server
- Supabase MCP Server

# MCP — The Orchestration Layer

1

**AI Agent**

Claude Code / Gemini CLI

2

**MCP**

Model Context Protocol

*The orchestration layer*

**Data**

Firebase MCP / Supabase MCP

**Identity**

Supabase Auth MCP

**API**

Stripe MCP / GitHub MCP

**Realtime**

Supabase Realtime MCP

**IoT**

(coming soon)

# MCP in Action — Stripe Example

## Before MCP

- Open the Stripe dashboard in the browser
- Manually create a product and a price
- Configure the webhook endpoint
- Copy API keys into the Flutter project
- Manually install the Stripe package
- Write the integration code in the Repository

## After MCP

> *"Create a monthly subscription product at €9.99 and configure the webhook for my Flutter app"*

- ✅ Agent creates the product in Stripe
- ✅ Agent configures the price
- ✅ Agent generates the webhook
- ✅ Agent installs the Flutter package
- ✅ Agent generates the Repository code

# Firebase Integration — Flutter + MCP

## Prerequisites

- Node.js + npm installed
- Firebase CLI installed
- firebase login run in terminal

## MCP Configuration — once

```
{
  "mcpServers": {
    "firebase": {
      "command": "npx",
      "args": ["-y", "firebase-tools@latest", "mcp"]
    }
  }
}
```

## What the Agent Can Do

- ✅ Create the Firebase project
- ✅ Configure Firestore + security rules
- ✅ Configure Firebase Auth
- ✅ Download and inject the SDK config into Flutter

# WHAT'S NEXT

**TP4**

Continue your project & How to Connect a MCP

CM5

Testing - Security - Deployment